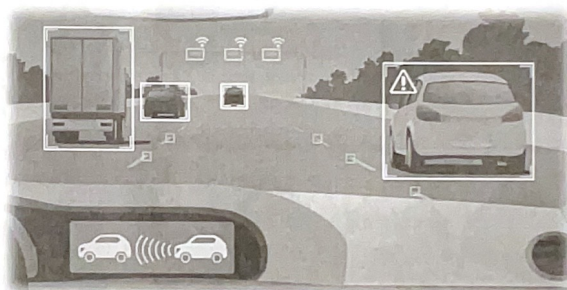


## PROTOTYPAGE DE L'ASSERVISSEMENT D'UN VEHICULE AUTONOME

### Compétences:

- ☐ Identifier la structure d'un système asservi.
- ☐ Associer un modèle aux composants des chaînes fonctionnelles.
- ☐ Proposer une démarche permettant de déterminer des grandeurs électriques.
- ☐ Intégrer les composants des chaînes fonctionnelles dans un prototype.
- ☐ Implémenter et exécuter un programme sur une cible.



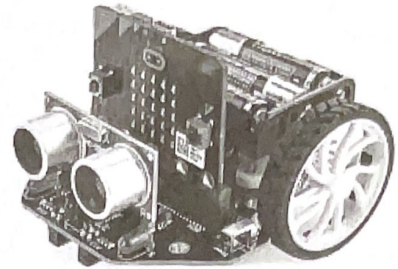
### MATERIEL

- 1 robot macqueen
- 3 piles 1.5V
- 1 shield pour carte micro:bit
- résistances  $10k\Omega$  et  $4.7k\Omega$
- 1 plaque de prototypage
- 1 câble usb
- 1 câble groove
- 6 fils de connexion

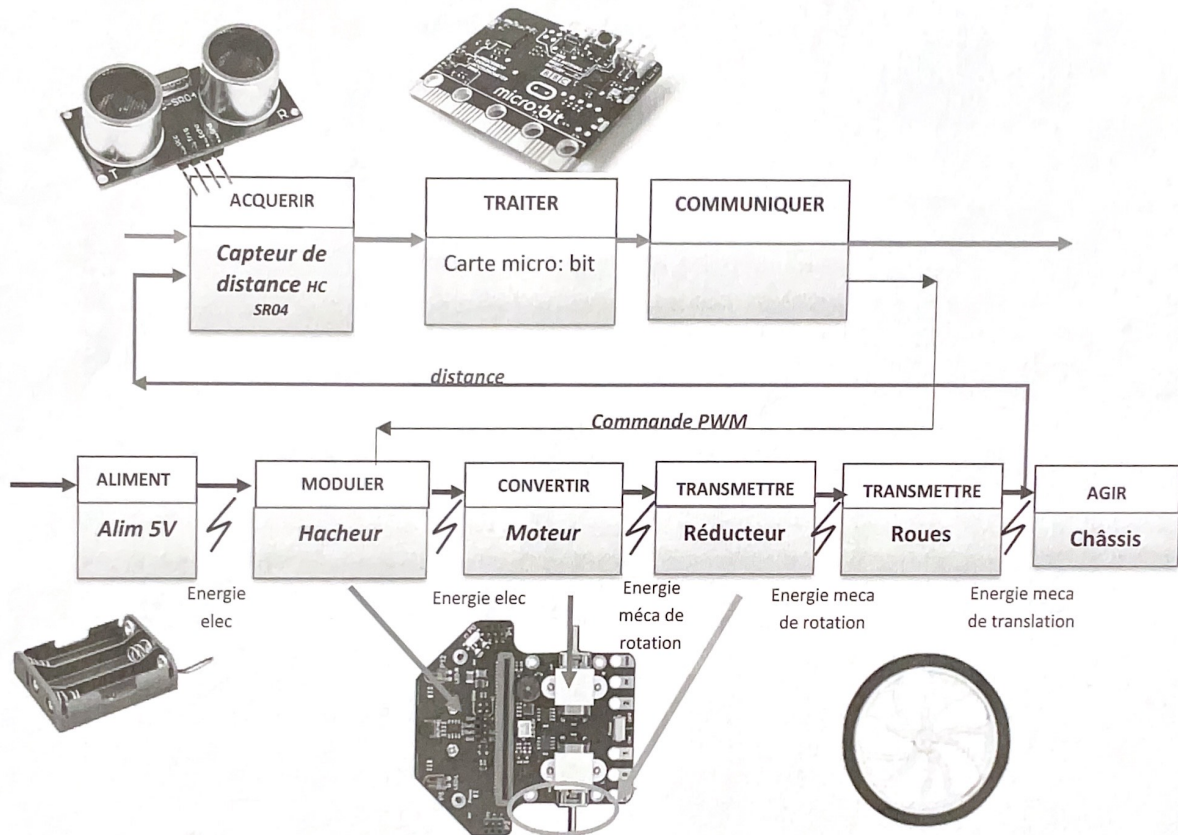
### Objectifs :

L'objectif est de mettre en place un système permettant à un véhicule de s'arrêter à une distance donnée d'un obstacle, ou de suivre un autre véhicule en conservant un espace avec ce dernier.

On propose de réaliser une maquette à échelle réduite.

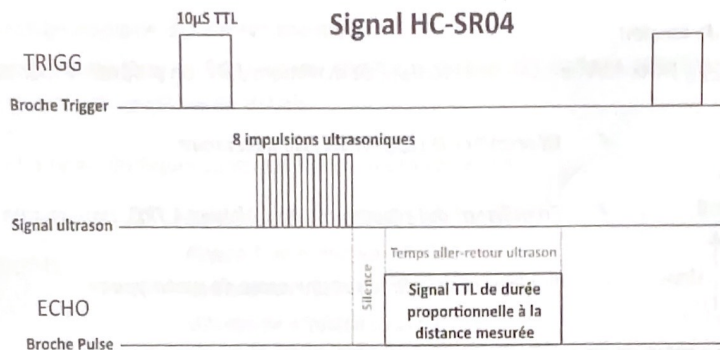


Voici les chaînes fonctionnelles de la maquette proposée :



## 1 PARTIE 1 : MESURE DE DISTANCE — MISE EN ŒUVRE DU CAPTEUR A ULTRA SON HC-SR04

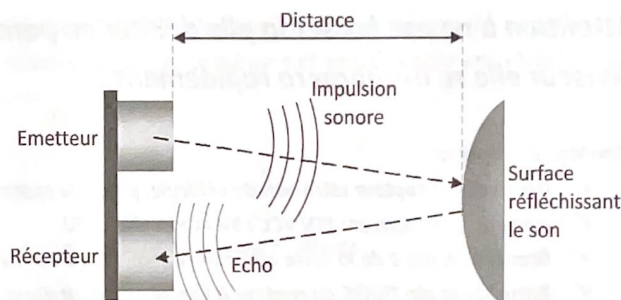
### 1.1 Principe de fonctionnement du capteur



Le principe de fonctionnement du capteur est basé sur une mesure du temps que met le son pour faire l'aller retour entre le capteur et l'obstacle.

Voilà comment se déroule une prise de mesure :

1. On envoie une impulsion HIGH de 10µs sur la broche TRIGGER du capteur.
2. Le capteur envoie alors une série de 8 impulsions ultrasoniques à 40 kHz (inaudible pour l'être humain, c'est quand plus agréable qu'un biiiiiiiip).
3. Les ultrasons se propagent dans l'air jusqu'à toucher un obstacle et retournent dans l'autre sens vers le capteur.
4. Le capteur détecte l'écho et clôture la prise de mesure.
5. Le signal sur la broche ECHO du capteur reste à HIGH durant les étapes 3 et 4, ce qui permet de mesurer la durée de l'aller-retour des ultrasons et donc de déterminer la distance.



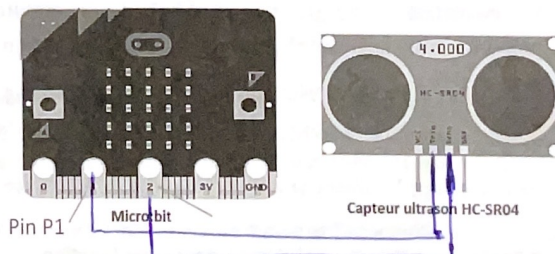
*N.B. Il y a toujours un silence de durée fixe après l'émission des ultrasons pour éviter de recevoir prématurément un écho en provenance directement du capteur.*

### 1.2 Câblage et alimentation

On utilisera les pins P1 et P15 de la carte micro:bit pour connecter respectivement les pins TRIGG et ECHO.

La pin P1 (signal TRIGG) sera donc une **sortie** pour la carte micro:bit (le signal sera émis de la carte micro:bit, vers le capteur)

La pin P15 (signal ECHO) sera donc une **entrée** pour la carte micro:bit (le signal venant du capteur est reçu pour être lu par la carte)



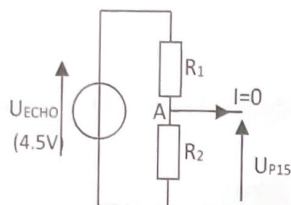
La carte micro:bit BBC est prévue pour fonctionner avec des entrées/sorties sur 3.3V, alors que notre capteur fonctionne en 5V. Le capteur doit donc être alimenté en 5V (on utilisera le bloc de piles 3\*1.5V fourni).

Il n'est pas gênant que la carte envoie un signal de 3.3V sur le TRIGG.

En revanche, le signal de 5V de l'ECHO, ne peut être envoyé directement sur l'entrée 3.3V de la carte, sous peine de risque de détériorer celle-ci. La plage admissible de tension d'entrée d'une broche de la carte microbit est  $V \in [2.8V, 3.8V]$ .

### Montage en pont diviseur de tension

Pour abaisser la tension  $U_{ECHO}$  (max 4.5V) en une tension  $U_{P15}$  de maximum 3.8V, on propose le montage suivant :



- ✓ Déterminer le rapport  $R_1/R_2$  nécessaire
- ✓ En utilisant des résistances de  $10k\Omega$  et  $4.7k\Omega$ , proposer un montage
- ✓ Réaliser ce montage sur une carte de prototypage

- ✓ Test : alimenter le pont diviseur avec un potentiel de 4.5V continu (en utilisant les piles du véhicule maqueen) : Vérifier qu'on obtient bien un potentiel compris entre 2.8V et 3.8V au point A.

**Attention à ne pas laisser la pile débiter en permanence dans le montage en pont diviseur elle se déchargera rapidement !**

### Montage du capteur

- ✓ Démonter le capteur ultra-son du véhicule, placer le capteur sur la carte de prototypage (à un emplacement libre)
- ✓ Alimenter ce capteur : PIN VCC : 5V et PIN GND : 0V
- ✓ Brancher la pin 1 de la carte microbit (signal TRIGG envoyé par la carte) sur le capteur
- ✓ Brancher la pin TRIGG du capteur à l'entrée du pont diviseur
- ✓ Brancher la pin 15 de la carte microbit (signal ECHO à lire par la carte) sur le pont diviseur
- ✓ Bien raccorder les masses de la carte et de l'alimentation (pile)
- ✓ Il doit vous rester un fil rouge (alimentation de la carte) non raccordé : la carte sera alimentée par l'USB de l'ordinateur (qui servira de toute façon à implanter le programme)

**Faire valider votre branchement par le professeur : vous connecterez 1 fil à vide sur le point A**

### 1.3 Programmation de la carte pour mesure de la distance

`pin1.write_digital(1)` : instruction permettant de mettre à 1 (état haut) la sortie pin1

`pin1.read_digital()` : instruction permettant de lire l'état de l'entrée sur pin1

`t= machine.time_pulse_us(pin1,1)` : mesurer la durée t pendant laquelle pin1 est à l'état haut (1) (importer avant la bibliothèque **machine** avec l'instruction : `import machine`)

`utime.sleep_us(t)` : attendre t (t en microsecondes)

(importer avant la bibliothèque **utime** avec l'instruction : `import utime`)

`print((m,))` : affiche la valeur m sur le REPL et la trace sur le graphique

- ✓ A l'aide des instructions ci-dessus, écrire une fonction `mesure()`, qui retourne (en m) la distance entre l'objet et le capteur.
- ✓ Afficher toutes les 0.5 secondes le résultat de cette mesure.

**Rappel : pour afficher sur le REPL en temps réel :**

- Téléverser votre programme sur la carte (Flasher)
- Cliquer sur **Graphique**
- Puis cliquer sur **REPL**

### 1.4 Validation

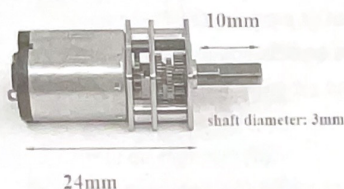
- ✓ Mettre en place un protocole permettant de valider la mesure.
- ✓ Conclure sur la linéarité, la justesse, le domaine de validité de la mesure de distance.

## 2 PARTIE 2 : PILOTAGE DE LA VITESSE : ETUDE DE LA COMMANDE MOTEUR, MODELISATION DU COMPORTEMENT DE L'ENSEMBLE MOTOREDUCTEUR CHASSIS

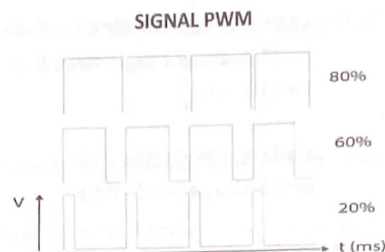
### 2.1 Principe de fonctionnement du driver moteur

Le principe de pilotage du motoréducteur est basé sur une communication en I2C entre la carte microbit et le driver (hacheur) implanté dans la carte électronique du châssis.

Le constructeur annonce les caractéristiques suivantes pour le moto-réducteur :



**Puissance nominale :** 0.5W  
**Rapport de réduction :** 1 : 150  
**Rendement :** 0.9  
**Vitesse de rotation sous 5V :** 133 tr/min  
 (en sortie de réducteur)  
**Mode de pilotage en PWM**



Le pilotage en PWM consiste à envoyer au moteur une tension entre 0 et 5V hachée sur une période très courte. La période des crêteaux étant si faible que le moteur reçoit la tension moyenne du signal, ce qui permet ainsi d'obtenir une variation de la tension à ses bornes, ainsi une variation de vitesse du moteur.

**Le pilotage de chaque moteur est indépendant.**

### 2.2 Programmation du driver

Fonctions python permettant le pilotage du moteur en fonction des deux variables **sens** et **vitesse**.

```
def moteurDroit(sens, vitesse):
    i2c.write(0x10, bytearray([2, sens, vitesse]))

def moteurGauche(sens, vitesse):
    i2c.write(0x10, bytearray([0, sens, vitesse]))
```

La variable **sens** prend pour valeur **0** pour la marche avant ou **2** pour la marche arrière.

La variable **vitesse** est une valeur numérique codée sur 8 bits, elle varie donc entre 0 et 255.

- ✓ A l'aide des instructions en annexe 1, implémenter le programme permettant la mise en route des moteurs pour une vitesse de 125.
- ✓ A l'aide du tachymètre, mesurer la vitesse obtenue de la roue droite, la vitesse expérimentale correspond-elle à la vitesse théorique ?
- ✓ Reprendre pour un PWM de 80, 180, 255.

### 2.3 Estimation de la phase d'accélération

L'objectif de l'étude qui va suivre est d'estimer la distance parcourue lors de la phase d'accélération.

Dans un premier temps, à partir de la chaîne d'énergie de la transmission,

- ✓ Déterminer le couple d'entraînement à la sortie du réducteur lorsque la vitesse du moteur est maximum.

Par une étude dynamique de l'ensemble, en négligeant la résistance au roulement, la masse de l'ensemble {véhicule + carte} est de 130g :

- ✓ Estimer par le calcul l'accélération du véhicule.
- ✓ Quelle est alors la durée de la phase d'accélération ?

## 2.4 Validation avec mesure distance parcourue-temps

✓ Implémenter le programme qui :

- lors de l'appui sur le bouton A, déclenche la rotation des 2 moteurs à un PWM de 125, pendant environ 8 secondes
- inclut la mesure de la distance de la voiture à un obstacle
- envoie l'information de distance par radio toute les 0.1 secondes

```
import radio #Import de la bibliothèque
radio.config(group=99) #Configuration de la radio (ligne à placer en début de programme)
radio.on() #Mise en route de la transmission (ligne à placer en début de programme)

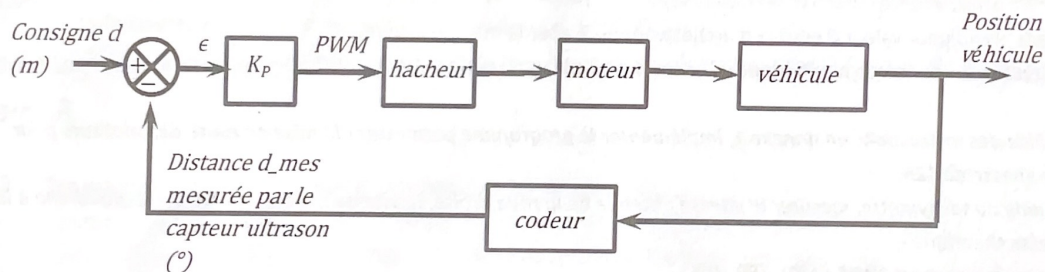
radio.send(str(v)) #Envoi d'un message par radio
m=radio.receive() #Réception d'un message par radio
```

On utilisera une autre carte, branchée sur l'ordinateur et connectée à mu, pour recevoir les informations envoyées par radio.

- ✓ Valider par un tracé graphique la linéarité de la courbe (temps de parcours, distance à l'obstacle)
- ✓ Comparer vitesse mesurée et vitesse prévue par le modèle

## 3 PARTIE 3 : MISE EN PLACE D'UN ASSERVISSEMENT

On propose de mettre en place la structure d'asservissement suivante afin que le robot macqueen se place systématiquement à une distance  $d$  (par exemple 10cm) de l'obstacle devant lui.



On doit écrire le programme qui calculera la consigne PWM en fonction de la distance mesurée par le capteur.

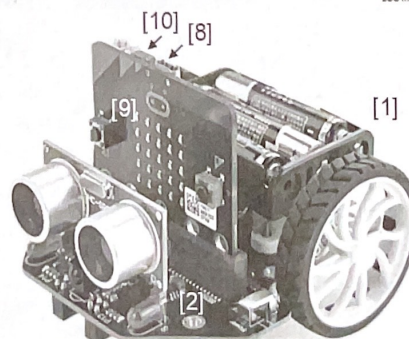
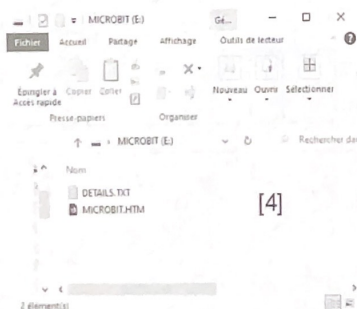
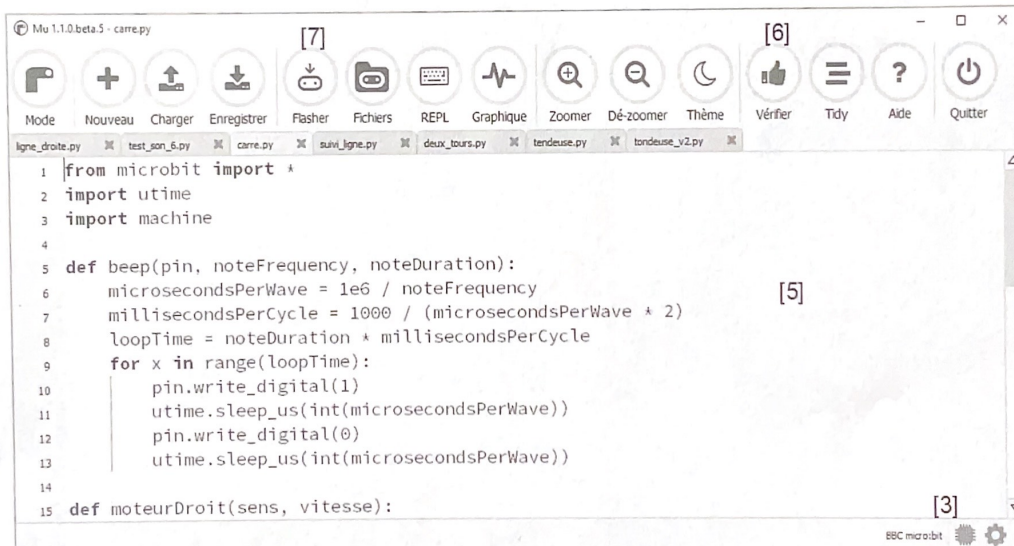
- ✓ A partir du schéma bloc ci-dessus :
  - Ecrire l'expression de l'écart  $\epsilon$  en fonction de  $d$  et  $d_{mes}$
  - Ecrire la relation entre la valeur du PWM et l'écart.
- ✓ Calculer la valeur de  $K_p$  qui permet d'obtenir la vitesse maximale du moteur quand l'écart est de 0.5 m
- ✓ Réfléchir aux 2 cas suivants : l'écart est supérieur à 0.5 m ; l'écart est négatif
- ✓ Programmer l'asservissement sur votre carte micro:bit, tester sur le véhicule

## Annexes :

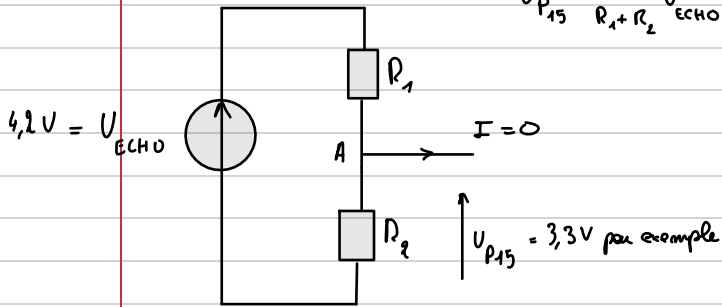
**Lien pour installer le logiciel mu :** <https://codewith.mu/en/download>

### Prise en main Maqueen

- ✎ Lancer le logiciel Mu Editor
- ✎ Mettre **hors tension** le robot avec l'interrupteur ON/OFF à l'arrière [1]. La del « Power » indique que le robot est sous tension [2].
- ✎ Connecter le Maqueen au PC avec le câble USB, l'indication en bas à droite permet de vérifier que le robot est bien connecté [3]. Vous pouvez fermer la fenêtre Windows confirmant la présence de la carte si elle s'ouvre [4].
- ✎ Le programme est écrit dans la fenêtre principale [5].
- ✎ Il est possible de vérifier les erreurs de syntaxe avec l'outil « Vérifier » [6].
- ✎ L'outil « Flasher » [7] permet de copier le programme sur la carte micro bit. Attendre que la del à l'arrière de la carte arrête de clignoter [8].
- ✎ Si un message s'affiche sur la matrice de del, le programme ne fonctionnera peut-être pas à cause d'une ou plusieurs erreurs [9].
- ✎ Débrancher le câble USB et mettre le robot sous tension. Le programme s'exécute immédiatement.
- ✎ Observer le fonctionnement et mettre le robot hors tension.
- ✎ Il est possible d'appuyer sur le bouton « reset » à l'arrière de la carte [10] pour relancer le programme et arrêter le robot.
- ✎ Si vous devez attraper le robot en mouvement soyez très précautionneux. Il faut le soulever par le châssis sans toucher aux roues ni à l'électronique.



$$U_{P_{15}} = \frac{R_2}{R_1 + R_2} U_{\text{ECHO}} \Leftrightarrow 3,3 = \frac{4,2 R_2}{R_1 + R_2} \Leftrightarrow \frac{3,3}{4,2} = 0,78 \text{ donc } \frac{R_2}{R_1 + R_2} \simeq 0,8$$



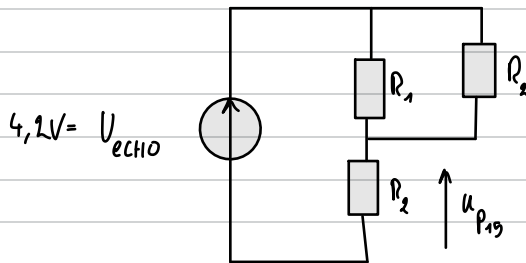
Il faut trouver les valeurs de  $R_2$  et  $R_1$  pour avoir un rapport de 0,8.

avec  $R_1 = 4,7 \text{ k}\Omega$   
 $R_2 = 10 \text{ k}\Omega$ 
 $\frac{10}{10 + 4,7} = \frac{10}{14,7} = 0,68 \rightarrow \text{pas assez.}$

On ajoute une résistance en parallèle à  $R_1$  de  $10 \text{ k}\Omega$  pour tester :  $R_{eq} = \frac{4,7 \times 10}{4,7 + 10} \simeq 3 \text{ k}\Omega$

On obtient  $\frac{R_2}{R_{eq}} = \frac{10}{13} \simeq 0,75 \text{ k}\Omega \rightarrow \text{proche}$

On aura le montage suivant



Tension mesurée de 3,16V.